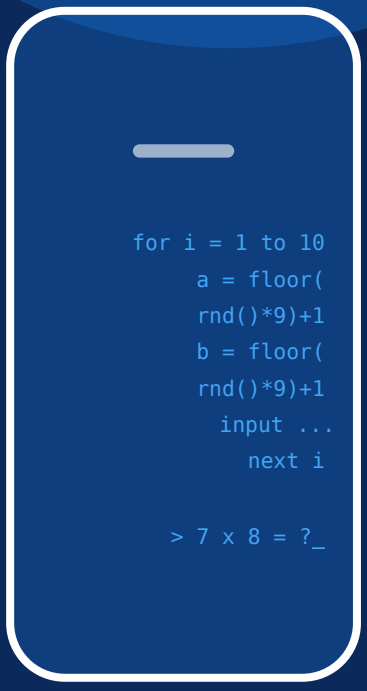
PROGRAMAÇÃO DE BOLSO RFO BASIC

AULA

03

Quiz



Seu celular vira juiz de um quiz: sorteia contas, confere respostas e conta pontos.

RECURSO DO CELULAR

Sorteio e teclado

laço for · rnd() · acumulador · if dentro de laço

Projeto de Ensino em Programação · Novembro de 2026

NESTA AULA VOCÊ VAI

- ✓ Repetir tarefas com o laço `for` e o seu **contador**.
- ✓ Sortear números inteiros em qualquer intervalo com `rnd()`.
- ✓ Acumular **pontos** numa variável e calcular uma porcentagem.
- ✓ Colocar um `if` **dentro** de um laço.
- ✓ Construir seu primeiro jogo completo, com começo, meio e fim.

Até aqui o celular executou cada instrução uma vez. Se quiséssemos 10 perguntas de multiplicação, teríamos de copiar o mesmo bloco 10 vezes, e qualquer correção teria de ser feita 10 vezes. Hoje aprenderemos a pedir ao celular que **repita**, e usaremos isso para criar uma competição de tabuada.

1 O laço for

O `for` repete um bloco um número **conhecido** de vezes. Teste:

Listagem 1: Laço `for` com cinco iterações

`laco-for.bas` `basic`

```
1  for i = 1 to 5
2    print "Rodada: " + int$(i)
3  next i
4
5  end
```

```
Rodada: 1
Rodada: 2
Rodada: 3
Rodada: 4
Rodada: 5
```

RODAR Execute e depois troque o `5` por `8`. Só um número mudou, mas o programa repetiu mais vezes.

A variável `i` é o **contador**. Ela começa em `1`, vale `2` na rodada seguinte, e assim por diante, até `5`. O que estiver entre `for` e `next` é o **corpo** do laço e é executado uma vez para cada valor do contador.

CONCEITO · LAÇO FOR

`for contador = início to fim` repete o corpo para cada valor do contador, de `início` até `fim`. O `next` fecha o laço. Com `step`, o contador pode pular de 2 em 2 (`step 2`) ou contar para trás (`step -1`).

Listagem 2: Laço `for` com cinco iterações decrescentes

laco-for-step.bas — basic

```
1  tts.init
2
3  for i = 5 to 1 step -1
4    tts.speak int$(i)
5    pause 500
6  next i
7
8  tts.speak "Asas batendo,"
9  tts.speak "Marcha de decolagem,"
10 tts.speak "Turbinas e"
11 tts.speak "Já"
12 tts.stop
13
14 end
```

RODAR Execute a contagem. Depois troque o 5 por 10 e o `step -1` por `step -2`.

O **contador** começa em 5 e diminui de 1 em 1 (`step -1`) até chegar a 1. O programa entra no laço e executa todo o código até `next`, dizemos então que ocorreu uma iteração ou uma volta/rodada. Quando todas as iterações acabarem, o programa sai do laço.

Acumuladores

Repetição fica poderosa quando uma variável **acumula** resultados. Este laço soma os números de 1 a 10:

Listagem 3: Laço `for` com acumulador

acumulador.bas — basic

```
1  soma = 0
2
3  for i = 1 to 10
4    soma = soma + i
5  next i
6
7  print "Soma: " + int$(soma)
8
9  end
```

Como saber o que acontece dentro do laço? Simulamos o programa no papel, linha a linha.

Volta	Valor de `i`	`soma` no fim da volta
1	1	0 + 1 = 1
2	2	1 + 2 = 3
3	3	3 + 3 = 6
4	4	6 + 4 = 10
…	…	…
10	10	45 + 10 = 55

CONCEITO · ACUMULADOR

É uma variável que começa com um valor inicial (aqui, `soma = 0`) e é atualizada a cada volta. A linha `soma = soma + i` lê assim: “pegue o valor atual de `soma`, some `i` e guarde de volta em `soma`”.

RODAR Confira o resultado.

ATENÇÃO

O valor inicial do acumulador tem de ser definido **antes** do laço. Se `soma = 0` ficasse dentro do corpo, a soma voltaria a zero a cada volta.

2 Sorteando números

Um quiz precisa de perguntas diferentes a cada rodada. Como vimos na aula passada, o comando `rnd()` sorteia um número **decimal** qualquer maior ou igual a 0 e menor que 1, como 0.2749 ou 0.9031. Para obter inteiros, vamos **transformar** esse número com expressões:

Expressão	Valores possíveis
`rnd()`	qualquer decimal de 0 até quase 1
`rnd() * 9`	qualquer decimal de 0 até quase 9
`floor(rnd() * 9)`	inteiros 0, 1, 2, …, 8
`floor(rnd() * 9) + 1`	inteiros 1, 2, 3, …, 9

O `floor` arredonda para baixo, e o `+ 1` desloca o intervalo. Isso vale para qualquer faixa.

3 Quiz da tabuada

Uma pergunta só

Antes de repetir, fazemos funcionar **uma** rodada. Esta é a versão mínima do jogo:

Listagem 4: Quiz simples

`quiz.bas` `basic`

```
a = floor(rnd() * 9) + 1
b = floor(rnd() * 9) + 1
input int$(a) + " x " + int$(b) + " = ", resp

if resp = a * b
  print "Acertou!"
else
  print "Errou!"
endif

end
```

O que acontece aqui:

1. `a` e `b` recebem números sorteados de 1 a 9.
2. O `input` mostra a conta **montada com texto** e guarda a resposta do jogador em `resp`.
3. O `if` compara `resp` com o produto `a * b`.

RODAR Execute algumas vezes. As contas mudam a cada execução? Acerte uma, erre outra.

Dez perguntas, um placar

Agora a ideia central: **envolver a rodada num laço**, acrescentar um acumulador de acertos e um resultado final.

Parte	Decisão
Repetição	`for i = 1 to total`, em que `total = 10`.
Estado	`acertos`, que começa em 0 e sobe 1 a cada resposta certa.
Corpo do laço	Sortear, perguntar, conferir, atualizar o placar.
Depois do laço	Calcular a porcentagem e comentar o resultado.

Listagem 5: Quiz com 10 perguntas e placar

for-quiz.bas basic

```
1   acertos = 0
2   total = 10
3
4   for i = 1 to total
5     a = floor(rnd() * 9) + 1
6     b = floor(rnd() * 9) + 1
7     input int$(a) + " x " + int$(b) + " = ", resp
8
9     if resp = a * b
10      acertos = acertos + 1
11      print "Acertou!"
12    else
13      print "Errou!"
14    endif
15  next i
16
17  porcentagem = round(acertos / total * 100)
18  print "Você acertou " + int$(acertos) + " de " + int$(total);
19  print " (" + int$(porcentagem) + "%)"
20
21  end
```

```
Acertou!
Errou!
Acertou!
…
Você acertou 8 de 10 (80%)
```

Repare em como as ideias se combinam: o `for` controla **quantas vezes**; as variáveis `a` e `b` são **redefinidas** a cada volta, por isso cada pergunta é nova; e `acertos` **sobrevive** de uma volta para outra, porque nunca é zerada dentro do laço.

RODAR Execute o jogo completo. Anote suas respostas e verifique o placar final com um teste de mesa no papel.

DICA

A porcentagem usa a expressão `round(acertos / total * 100)`. O `round` arredonda para o inteiro mais próximo. Sem ele, 7 de 9 apareceria como `77.77777...`.

4 Quebre o código!

Mude **uma coisa de cada vez** e observe o que acontece:

QUEBRAR Mova a linha `acertos = 0` para dentro do laço, logo depois do `for`. Qual é o placar final? Por quê?

QUEBRAR Mude `total = 10` para `total = 3`. Todas as partes do programa se ajustaram sozinhas? Por quê?

QUEBRAR Troque o `9` por `99` nas duas linhas de sorteio. O jogo ficou mais difícil? E mais divertido?

DICA

O quarto experimento mostra a vantagem de usar variáveis como `total` em vez de repetir o número `10` em vários lugares: mudar o programa exige alterar **um único ponto**. Você pode modificar o programa para que o usuário diga quantas rodadas ele deseja jogar, basta usar `input` para a variável `total`.

DESAFIO

Duelo de dois jogadores

Transforme a competição em um **duelo** no mesmo celular:

1. pergunte o nome dos dois jogadores (`nome1$` e `nome2$`);
2. as perguntas se alternam: **Jogador 1** nas voltas ímpares, **Jogador 2** nas pares;
3. cada jogador tem seu próprio placar;
4. no final, o celular **fala** quem venceu, ou anuncia o empate.

Pista: o resto da divisão descobre a vez de quem. A expressão `mod(i, 2)` vale 1 nas voltas ímpares e 0 nas pares:

```
if mod(i, 2) = 1
  ! vez do jogador 1
else
  ! vez do jogador 2
endif
```

Bônus 1: deixe o jogador escolher o **tamanho máximo** dos números (fácil até 9, difícil até 99) usando uma variável no lugar do `9`.

5 Resumo

Ideia ou comando	Para que serve
`for i = a to b ... next i`	Repete o corpo para cada valor de `i`, de `a` até `b`.

`step`	Muda o passo do contador (`step 2`, `step -1`).
Acumulador	Variável inicializada fora do laço e atualizada dentro dele.
Teste de mesa	Simular o laço no papel, volta por volta.
`if` dentro de `for`	A cada volta, o programa decide um caminho.
`mod(a, b)`	Resto da divisão de `a` por `b`.
`round(x)`	Arredonda para o inteiro mais próximo.

NA PRÓXIMA AULA

Aula 4: Super Quiz. E se não soubermos **quantas** perguntas vão ser feitas? O jogo só termina quando o jogador **errar**. Para isso conheceremos o laço `while`, que repete **enquanto** uma condição for verdadeira, e usaremos uma sequência de acertos para aumentar a dificuldade.